

Automatisierung der Datenextraktion - Entwicklung eines Web-Scrapers für die akademische Forschung

Moritz Gilges

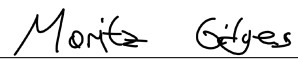
Bachelorarbeit

Beginn der Arbeit:	29. April 2025
Abgabe der Arbeit:	29. Juli 2025
Gutachter:	Prof. Dr. Stefan Conrad Prof. Dr. Michael Leuschel

Erklärung

Hiermit versichere ich, dass ich diese Bachelorarbeit selbstständig verfasst habe. Ich habe dazu keine anderen als die angegebenen Quellen und Hilfsmittel verwendet.

Düsseldorf, den 29. Juli 2025



Moritz Gilges

Zusammenfassung

Die vorliegende Bachelorarbeit beschäftigt sich mit der Entwicklung und Evaluation eines Web-Scrapers zur automatischen Extraktion spezifischer Daten aus dem Quellcode von Webseiten. Das System ist als lokale Webanwendung in der Programmiersprache Java mithilfe von Spring Boot entwickelt worden. Neben der reinen Extraktion der Daten können Nutzer:innen diese in einer Benutzeroberfläche bearbeiten und in einer Datenbank sichern. Darüber hinaus wird der Export dieser gesammelten Daten in CSV- und JSON-Formate ermöglicht. Der Fokus der Arbeit liegt bei der Extraktion von Daten aus Tabellen- und Listenstrukturen, da diese fast ausschließlich zur Datenaufbereitung verwendet werden. Dabei wird eine Vielzahl an Variationen betrachtet, um eine robuste und flexible Lösung zu schaffen. Des Weiteren wurde auf eine parallele Verarbeitung gesetzt, um eine hohe Performanz zu gewährleisten, insbesondere bei der Verarbeitung großer Datenmengen. Im Rahmen der Arbeit wurden umfangreiche Testszenarien entwickelt, um die Funktionalität des gesamten Systems zu prüfen. Besonders wurde sich dabei auf die korrekte Verarbeitung der verschiedenen Tabellen- und Listen-Variationen fokussiert, die den Kern der Anwendung bilden. Diese Arbeit beschreibt den Entwicklungsprozess, die zugrunde liegenden Entscheidungen sowie die technische Realisierung. Sie vereint verschiedene softwaretechnische Konzepte und bietet eine praktische Anwendungsmöglichkeit, insbesondere im Kontext der akademischen Forschung.

Inhaltsverzeichnis

1 Grundlagen	1
1.1 Web Scraping: Definition und Anwendung	1
1.2 Relevanz für die akademische Forschung	1
1.3 Rechtliche und ethische Aspekte des Web Scraping	2
1.4 Technologien und Frameworks	2
2 Anforderungsanalyse	4
2.1 Funktionale Anforderungen	4
2.2 Nicht-funktionale Anforderungen	4
2.3 Stakeholder und Zielgruppe	5
3 Konzeption und Design	5
3.1 Architektur des Systems	5
3.2 Datenmodell und Datenbankdesign	8
3.3 Frontend- und Backend-Integration	12
3.4 Sicherheitsaspekte	13
4 Implementierung	15
4.1 Implementierung des Web Scrapers	15
4.2 Frontend-Funktionen und Benutzeroberfläche	19
5 Evaluation	21
5.1 Controller- und Datenbank Tests	21
5.2 Scraping von Tabellen	21
5.3 Einschränkungen des Systems	29
6 Diskussion	30
6.1 Erfüllung der Anforderungen	30
6.2 Weiterentwicklungsmöglichkeiten	32
6.3 Fazit	32
Literatur	33
Abbildungsverzeichnis	35

Tabellenverzeichnis

35

1 Grundlagen

Um die Komplexität und generelle Funktionsweise des entwickelten Systems nachvollziehen zu können, ist ein grundlegendes Verständnis zentraler Konzepte und Techniken erforderlich. Dieses Kapitel liefert eine Einführung in die wesentlichen theoretischen und technischen Grundlagen, auf die diese Arbeit im weiteren Verlauf aufbaut.

1.1 Web Scraping: Definition und Anwendung

Web Scraping bezeichnet eine automatisierte Methode zur Extraktion von spezifischen Daten aus Webseiten, bei der HTML-Quellcode, systematisch analysiert und relevante Informationen in eine strukturierte Form wie Datenbanken überführt werden. Diese Technik findet vielseitig Anwendung, insbesondere bei Forschungsprojekten, die große Mengen an unterschiedlichen Daten für Analysen benötigen. So setzen u.a. Unternehmen Web Scraping häufig für Marktanalysen ein (Boegershausen et al., 2022). Folglich wird beschrieben: „it is important for organizations to know how to mine that data in order to extract useful knowledge.“ (Lotfi et al., 2021, S. 1). In der vorliegenden Arbeit wird Web Scraping genutzt, um akademische Forschungsprojekte durch die effiziente Sammlung und Aufbereitung von Daten zu unterstützen, die für wissenschaftliche Analysen erforderlich sind. Darüber hinaus wird eine Frontend-Schnittstelle bereitgestellt, welche das Ziel einer leichteren Bedienung ermöglichen soll und zusätzlich die Verwaltung der Daten beinhaltet.

1.2 Relevanz für die akademische Forschung

Besonders für die akademische Forschung bietet Web Scraping eine große Unterstützung, indem es Forscher:innen eine einfache Möglichkeit bereitstellt, große Mengen an Daten effizient zu sammeln. Insbesondere Daten, die nur mühsam zugänglich wären, z.B. wenn keine APIs vorhanden sind (Struck, 2025), würden sie ohne Web Scraping nicht erfasst werden. Daher bietet diese Methode für den Lehrstuhl für Datenbanksysteme zahlreiche neue Möglichkeiten und vereinfacht bereits bestehende Prozesse.

Ein zentraler Anwendungsfall stellt die Erhebung von Forschungsdatensätzen dar. Diese können als Grundlage für Experimente, Analysen und Entwicklung oder Weiterentwicklung von Datenbanktechnologien herangezogen werden. Gewonnene Daten können unter anderem für Benchmarks genutzt werden, um verschiedene Algorithmen oder Datenbankabfragen miteinander zu vergleichen. Ein anderer Nutzen besteht in der Erfassung von dynamischen Daten, in Form von etwa Echtzeit-Datenfeeds, die besonders relevant für Forschungsprojekte im Bereich Big Data und Echtzeit-Analysen sind.

Darüber hinaus kann Web Scraping die Lehre am Lehrstuhl unterstützen, indem es Student:innen ermöglicht wird, praxisnahe Erfahrungen im Umfeld mit realen Daten zu sammeln. Inbegriffen ist nicht nur die Integration von Web Scraping in studentischen Projekten, bei denen Datenextraktion, -aufbereitung und -speicherung eine zentrale Rolle spielen, sondern auch der Umgang mit größeren Datenmengen in der Anwendungsentwicklung.

Die Implementierung eines eigenen Web-Scraping-Systems bietet dem Lehrstuhl viel Flexibilität und Unabhängigkeit. Anstatt auf externe Dienste angewiesen zu sein, kann das System individuell auf die Bedürfnisse der Forschungsprojekte zugeschnitten werden, was auch die Möglichkeit einer spezifischen Konfiguration beinhaltet. Somit trägt Web Scraping nicht nur zur Erweiterung des Forschung Repertoires bei, sondern fördert auch die Innovation und Effizienz des Lehrstuhls.

1.3 Rechtliche und ethische Aspekte des Web Scraping

Neben den möglichen technischen Schwierigkeiten, birgt Web Scraping rechtliche sowie ethische Herausforderungen. Aus rechtlicher Sicht können Probleme durch die Verletzung von Urheberrechten oder Nutzungsbedingungen zustande kommen. In der Europäischen Union regelt insbesondere die DSGVO (Parliament und Council, [2016](#)) den Umgang mit personenbezogenen Daten, was die Nutzung dieser Technologie einschränken kann. Es liegt in der Verantwortung der Nutzer:innen, sich an diese Regeln zu halten.

In Bezug auf ethische Aspekte muss sichergestellt werden, dass kein Schaden durch z.B. übermäßige Serverbelastung oder die missbräuchliche Nutzung gesammelter Daten entsteht. Um dies besser gewährleisten zu können, bieten viele Webseiten sogenannte „robots.txt“-Dateien an. Diese sind oft direkt hinter der Wurzel-Url angehängt und führen bei Nichtbeachtung teilweise zu Sperrungen oder anderen Konsequenzen (Struck, [2025](#)).

1.4 Technologien und Frameworks

Bei der Entwicklung der Anwendung spielt die Wahl geeigneter Technologien eine Schlüsselrolle. Dabei sind vor allem Eigenschaften wie Effizienz, Robustheit und Wartbarkeit von großer Bedeutung. Die getroffene Wahl wird im Folgenden dargelegt und erläutert.

1.4.1 Spring Framework

Das Spring Framework (Contributors, [2025c](#)) bietet eine umfassende Grundlage für die Entwicklung von Web-Anwendungen, indem es vielseitige Funktionalitäten in einer einheitlichen Plattform bereitstellt. Durch die modulare Architektur und zentrale Konzepte wie Dependency Injection (Contributors, [2025b](#)) lässt sich die Entwicklung von gut wartbaren Code effizient realisieren. Ein weiterer Vorteil von Spring besteht in der Möglichkeit, die Anwendung in Zukunft als zentrale Webanwendung auszubauen, die mehrere Nutzer:innen gleichzeitig bedienen und durch Spring Security abgesichert werden kann.

In dem vorliegenden System finden vor allem die Module: Spring Web, Thymeleaf, Validation und Spring Data MongoDB eine breite Anwendung. Spring Web erleichtert den generellen Aufbau der Webanwendung erheblich, insbesondere indem eine klare Trennung von Controller- und View-Logik erfolgt. Die Darstellung der dynamischen Inhalte wird durch die serverseitige Templating-Engine Thymeleaf ermöglicht, die sich nahtlos in den Spring-Kontext integrieren lässt. Eine Alternative wäre ein clientseitiger An-

satz mit beispielsweise React, der die Logik stärker in das Frontend verlagert (Fadatare, 2024). Die in Spring enthaltenen Mechanismen zur Validierung tragen zu einer benutzerfreundlichen Oberfläche und einer sicheren Eingabeüberprüfung direkt beim Absenden von Formularen bei. Fehler durch invalide Eingaben werden Nutzer:innen direkt im Frontend mit einer passenden Fehlerbeschreibung angezeigt. Die Datenbankanbindung erfolgt mittels passender Treiber, sodass die Datenquellen mit wenig Aufwand ausgetauscht werden können. Auch beim automatisierten Testing bietet Spring Unterstützung, indem eine klare Trennung der Anwendungsschichten gewährleistet wird. Darüber hinaus bietet das Modul Spring Data MongoDB eine spezialisierte Variante für einfache und deklarative Datenbankzugriffe.

1.4.2 Docker

Um die Flexibilität und Konsistenz der verwendeten Technologien sicherzustellen, wurde auf Docker (Docker, 2025) zurückgegriffen. Docker ermöglicht die Containerisierung der gesamten Anwendung, was Abhängigkeiten, insbesondere die Datenbank, einschließt. Das ermöglicht identische Testumgebungen sowie eine problemlose Bereitstellung der Anwendung auf verschiedensten Plattformen und damit einhergehend eine einfache Skalierbarkeit.

1.4.3 MongoDB

Da es sich bei den extrahierten Daten des Web Scraping um prinzipiell heterogene und dynamisch strukturierte Inhalte handelt, wurde bewusst auf ein nicht-relationales Datenbanksystem zurückgegriffen. Mit MongoDB (MongoDB, 2025) fiel die Wahl auf eine weit verbreitete NoSQL-Datenbank, die sich besonders für flexible Daten eignet. Anders als relationale Systeme wird von MongoDB kein festes Schema verlangt, was sich ideal für die hier beschriebenen Daten eignet. Andere Vorteile einer relationalen Datenbank, wie die Fremdschlüssel-Beziehung, können deshalb hier nicht verwendet werden. Zusätzlich zu den technischen Aspekten, existieren zahlreiche Tools für MongoDB, wie etwa MongoDB Compass, welches zur Visualisierung der Daten verwendet werden kann.

1.4.4 Selenium

Für die technische Implementierung des Web Scraping wurde Selenium (Selenium, 2025) gewählt. Im Gegensatz zu anderen bekannten Bibliotheken wie JSOUP, die lediglich auf die Analyse statischer HTML-Inhalte spezialisiert sind, ermöglicht Selenium die Interaktion mit dynamisch generierten Webseiten. Das ist vor allem bei modernen Webseiten enorm wichtig, die JavaScript zum Laden dynamischer Inhalte verwenden. Zusätzlich kann Selenium automatisierte Interaktionen umsetzen, da es JavaScript-Code ausführen kann. In Kombination mit den komplexen Einstellungen, die Selenium bietet, können dadurch echte und realistische Nutzerszenarien simuliert werden.

1.4.5 Zusammenfassung

Die Kombination der aufgeführten Technologien bietet eine robuste, flexible und skalierbare Basis für die Entwicklung eines Web Scraping-Systems. Jedes der eingesetzten Tools trägt durch individuelle Stärken zur erfolgreichen Umsetzung des Projekts bei, indem die gewünschten Eigenschaften, siehe: Abschnitt 1.4, sichergestellt werden.

2 Anforderungsanalyse

Die Anforderungsanalyse beschreibt die spezifischen Aufgaben und Eigenschaften, die das System erfüllen soll, um den gesetzten Anforderungen zu entsprechen. Im Folgenden werden dafür funktionale als auch nicht-funktionale Anforderungen genauer betrachtet, sowie eine Analyse der Stakeholder und Zielgruppe durchgeführt.

2.1 Funktionale Anforderungen

Bei den funktionalen Anforderungen werden die grundlegenden Funktionalitäten des Web Scraping-Systems betrachtet. Das System sollte in der Lage sein, Daten aus strukturierten Inhalten, wie Tabellen oder Listen, verschiedenster Form effizient und sauber zu extrahieren. Dabei sollten nur die reinen Daten enthalten sein und kein anderer HTML-Quellcode. Aufgrund der enormen Vielzahl verschiedenster Formen kann das System keine Garantie gewährleisten, dass wirklich alle Daten gefunden werden, es soll zumindest aber die allermeisten und häufigsten Anwendungsfälle abdecken.

Darüber hinaus wird eine benutzerfreundliche Oberfläche bereitgestellt, über die das Web Scraping generell auf zwei Arten durchgeführt werden kann. Die Nutzung kann entweder manuell erfolgen, indem Nutzer:innen eine URL eingeben und eine Vorschau der extrahierten Daten direkt angezeigt bekommen, oder automatisiert, wobei zuvor bestimmte Konfigurationen festgelegt werden müssen. Die extrahierten Ergebnisse kann man anschließend in einer Datenbank sichern und in der Oberfläche anpassen. Darüber hinaus wird der Export der Daten in gängige Formate wie CSV und JSON ermöglicht, um sie flexibel für unterschiedliche Analysen zu verwenden.

2.2 Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen bestimmen die angestrebten Qualitätsmerkmale der Anwendung. Hierbei steht besonders die Wartbarkeit im Vordergrund. Zusätzlich muss das System möglichst effizient arbeiten, damit die teilweise großen Datenmengen in angemessener Zeit verarbeitet werden können. Weiterhin ist die Skalierbarkeit entscheidend, damit das System nachher auf eine Multi-User-Anwendung erweitert werden kann und trotzdem stabil und korrekt arbeitet. Das System soll trotz komplexer Vorgänge robust gegen interne und externe Fehlerquellen abgesichert sein.

2.3 Stakeholder und Zielgruppe

Die Anwendung ist spezifisch auf eine Zielgruppe zugeschnitten. Dabei sind die Hauptnutzer:innen des Systems akademische Forscher:innen und Entwickler:innen, die großen Datenmengen aus dem Internet benötigen. Ihre Anforderungen an Benutzerfreundlichkeit, Konfigurationsfreiheit und Zuverlässigkeit bestimmen die Ausgestaltung der Funktionalitäten. Deshalb wird unter anderem die Möglichkeit bereitgestellt spezifische Aspekte in den Einstellungen zu verändern, die ein gewisses technisches Know-How voraussetzen. Alle Aspekte der Anwendung sind trotzdem durch ausführliche Anleitungen in der Oberfläche erklärt, um eine reibungslose Nutzung sicherzustellen. Die Einhaltung von rechtlichen und ethischen Aspekten und die Rücksichtnahme auf die Serverkapazität der Zielseiten müssen dabei von Nutzer:innen berücksichtigt werden, vgl. Abschnitt 1.3.

3 Konzeption und Design

In diesem Kapitel wird die grundlegende Struktur der entwickelten Anwendung genauer beschrieben. Dabei werden zentrale Entwurfsentscheidungen, die Architektur, sowie das zugrunde liegende Datenmodell vorgestellt und erläutert. Außerdem wird auf die Integration von Frontend und Backend eingegangen, sowie Sicherheitsaspekte durch Mechanismen wie Validierung dargestellt.

3.1 Architektur des Systems

Bei der Architektur des Systems wurde sich für die Ports-And-Adapters (auch Hexagonale-) Architektur entschieden. Dieser Ansatz ist typisch für Webanwendungen und sorgt für eine klare Trennung der Verantwortlichkeiten und Abhängigkeiten zwischen den einzelnen Schichten, sodass diese unabhängig voneinander weiterentwickelt und getestet werden können. Diese Isolation bietet eine gute Grundlage für Wartbarkeit, ohne viel Komplexität in die Anwendung zu bringen.

Grundsätzlich wird bei der Architektur die Anwendung in drei Schichten: Application, Domain und Infrastructure eingeteilt.

Der Application-Layer beinhaltet alles, womit Benutzer:innen interagieren. In dem Fall dieser Anwendung sind dies die Controller, welche das Routing übernehmen und die Benutzeroberfläche bereitstellen, auf der alle Aktionen getätigt werden können. Anders formuliert: Der Application-Layer ist das Interface der Anwendung.

Der Domain-Layer hingegen ist der Kern der Anwendung und darf nur über bereitgestellte öffentliche Schnittstellen erreichbar sein. Grundlegend befindet sich hier die Geschäftslogik, welche durch eigene Interfaces von dem Application-Layer und dem Infrastructure-Layer erreicht werden kann. Außerdem werden hier die grundlegenden Entitäten, Werte und Aggregate abgebildet, also Objekte, die die Gegebenheiten der Realität darstellen.

Alle anderen Abhängigkeiten, die von der Anwendung benötigt werden, sind im Infrastructure-Layer vorhanden. Dieser beinhaltet meist, wie auch in dieser Anwendung, die Schnittstelle zu Datenbanken.

Damit der Infrastructure-Layer abhängig von dem Domain-Layer ist und nicht andersherum wird das Prinzip der Dependency-Inversion angewandt, indem wir über ein Interface mit dem Infrastructure-Layer kommunizieren (Ryś, 2024). Dadurch ist der Anwendungskern komplett unabhängig, was zu einer deutlich besseren Wartbarkeit und Testbarkeit führt.

In dieser Anwendung sind die einzelnen Schichten in verschiedenen Packages (Ordner für Klassen) untergebracht, um einen klaren strukturellen Überblick zu behalten. Die Gliederung dieser Architektur erfolgt hauptsächlich durch die Packages: `controller`, `database`, `domain` und `service`. In `controller` sind die jeweiligen Controller der Endpoints `/home/*`, `/manual/*`, `/automated/*` und `/settings/*` untergebracht, welche die jeweilige Funktionalität der Schnittstelle implementieren. Diese Controller-Klassen bekommen die benötigten Services automatisch über Spring injiziert, auf welche sie zugreifen, um Datenbankabfragen zu versenden. Diese Services greifen allerdings nicht direkt auf die Datenbank zu, da dies zu einer Verletzung der hexagonalen Architektur führen würde. Stattdessen bekommen sie automatisch die passende Implementierung eines eigenen Interfaces übergeben, welches die Schnittstelle genau festlegt. Diese Implementierung verwendet dann das Spring Data MongoDB-Interface, welches erweiterte CRUD-Anfragen bearbeiten kann. Außerdem wird hier teilweise zusätzlich das `MongoTemplate` verwendet, um sehr komplexe Anfragen umzusetzen, wie z.B. das Umbenennen und Löschen von einzelnen Feldern innerhalb des Daten-Segments. Um Konsistenzen einzuhalten, wurden reine Datenbankobjekte (DAOs) von Objekten aus dem Anwendungskern (domain models) getrennt. Die DAOs dienen ausschließlich der Kommunikation mit der Datenbank und werden nicht nach außen gegeben. Anstelle dessen werden sie direkt in die Domain-Objekte überführt (Mapping), welche dann weitergegeben werden. Durch diese Trennung wird Kopplung vermieden, was das Testen der Anwendung deutlich vereinfacht.

Neben dieser architektur-spezifischen Aufteilung befinden sich noch die beiden Packages `scheduler` und `scraper`. In `scheduler` wird das automatische Web Scraping initiiert und in `scraper` befinden sich die spezifischen Implementierungen der Datenextraktion, siehe Abschnitt 4.1. Eine genaue Übersicht ist in Abbildung 1 zu finden.

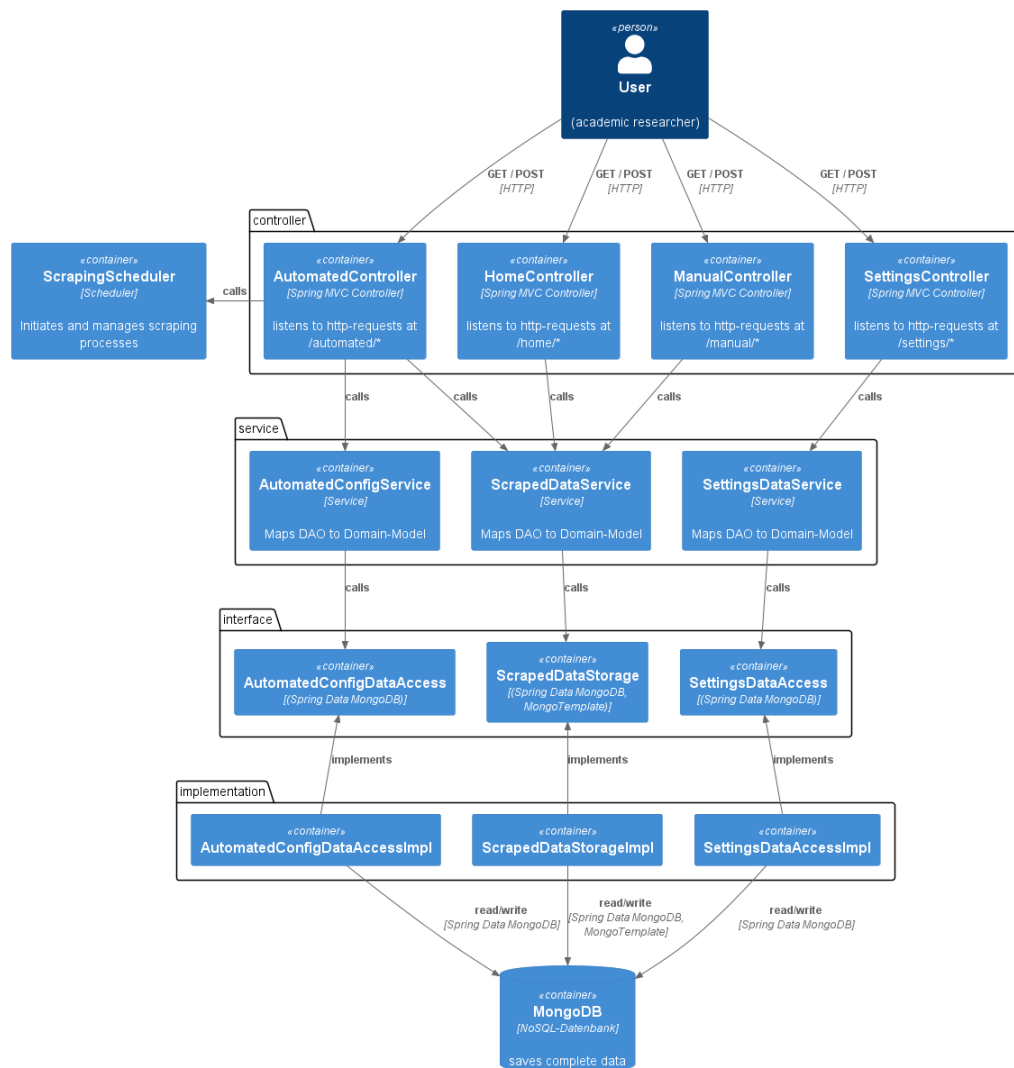


Abbildung 1: Hexagonale Architektur dieser Anwendung

3.2 Datenmodell und Datenbankdesign

Das Datenbankdesign der Anwendung ist so konzipiert, dass eine klare Trennung zwischen einzelnen Funktionalitäten geschaffen wird. Insbesondere wird dies durch das Modul Spring Data MongoDB gewährleistet, welches für alle Anfragen verwendet wird. Dieser Ansatz verfolgt das Prinzip hoher Wartbarkeit und Sicherheit, siehe: Abschnitt 3.4. Da das System mit verschiedenen Konfigurations-Daten und extrahierten Daten arbeiten muss, ergibt sich eine gewisse Komplexität, welche in diesem Kapitel ausführlich behandelt wird.

3.2.1 Einheitliches Datenmodell für extrahierte Daten

Die Flexibilität von MongoDB wurde für das zentrale Datenmodell der Anwendung gezielt genutzt. Sämtliche Daten des Web Scraping – unabhängig davon, ob sie manuell oder automatisiert extrahiert wurden – werden in einem einheitlichen Objekt vom Typ `ScrapedData` abgelegt. Dabei werden nicht nur die tatsächlichen Daten, sondern auch Metadaten angelegt, um relevante Informationen einfacher abrufen zu können, vgl. Tabelle 1.

Attributname	Datentyp	Beschreibung
id	String	Eindeutige ID im UUID-Format
dbName	String	Name der Datenbank
url	String	Quelle der extrahierten Daten
scrapedAt	LocalDateTime	Zeitpunkt des Scrapings
destination	Destination	Zuordnung der Tabelle
data	List<Map<String, String>>	Tatsächliche Informationen

Tabelle 1: Attribute der Klasse `ScrapedData`

Bei `Destination` handelt es sich um ein `enum`, welches die Werte *HOME* und *AUTOMATED* zugewiesen bekommen kann. Dieses Attribut ist sehr wichtig für die Anwendung, weil die automatisch extrahierten Daten grundsätzlich anders verwendet werden, siehe Abschnitt 3.2.3. Daher sollten sie in einer separaten Ansicht verwaltet werden, um eine gute Übersicht zu gewährleisten.

Insgesamt wird durch die Vereinheitlichung der extrahierten Daten nicht nur das Datenmanagement vereinfacht, sondern auch die Implementierung von Backend-Logik, da dieselben Verarbeitungsmethoden auf alle Datenobjekte angewendet werden können.

3.2.2 Datenrepräsentation

Die Darstellung der extrahierten Tabellendaten erfolgt intern als Auflistung von Schlüssel-Wert-Paaren, konkret in der Form einer `List<Map<String, String>>`. Dabei repräsentiert jede `Map` eine einzelne Tabellenzeile, wobei die Schlüssel den jeweiligen Spaltenüberschriften entsprechen, wie in Listing 1 und Tabelle 2 zu sehen.


```
List<Map<String, String>> data = List.of(
    Map.of("Name", "Alice", "Age", "24"),
    Map.of("Name", "Bob", "Age", "30")
);
```

Listing 1: Repräsentation als Java-Code

Name	Age
Alice	24
Bob	30

Tabelle 2: Visualisierung der zugehörigen Tabelle

Dieses Format wurde bewusst gewählt, da es eine hohe Flexibilität bei der Verarbeitung unterschiedlich strukturierter HTML-Tabellen bietet und eine einfache Verwendbarkeit erlaubt. Dabei lassen sich sowohl Tabellen mit variierender Spaltenanzahl als auch solche mit unregelmäßigen Header-Werten vereinheitlicht und verlustfrei abbilden. In Kombination mit MongoDBs dokumentenorientierter Speicherung ergibt sich eine äußerst flexible und anpassungsfähige Struktur der Datenhaltung, die auch dynamisch veränderte Webinhalte abdeckt.

Ein weiterer Vorteil dieser Struktur ist die einfache Verarbeitung im Frontend durch Thymeleaf und die leichte Überführung der Daten bei Exportvorgängen. Außerdem lassen sich einzelne Spalten durch die Map-Struktur gezielt bearbeiten, ohne die Integrität der restlichen Daten zu beeinflussen.

3.2.3 Konfigurationsdaten

Neben den eigentlichen Scraping-Daten verwaltet das System des Weiteren Konfigurationsdaten, die das Verhalten des automatisierten Scraping definieren. Diese sind essentiell für den Scheduler, welcher diese Durchläufe (Tasks) anstößt und werden durch die Klasse `AutomatedConfig` repräsentiert, vgl. Tabelle 3.

Attributname	Datentyp	Beschreibung
id	String	Eindeutige ID im UUID-Format
status	Status	Entscheidet, ob dieser Task aktiv ist
url	String	Quelle der extrahierten Daten
timeframe	Timeframe	Abstand zwischen Durchläufen
start	String	Zeitpunkt des ersten Durchlaufs
dbName	String	Name der Verwaltungs-Datenbank
overwrite	Overwrite	Zwei separate Funktionsweisen

Tabelle 3: Attribute der Klasse `AutomatedConfig`

Ein wesentlicher Bestandteil der Konfiguration ist das `status`-Attribut, welches den Zustand der Aufgabe bestimmt und zwischen den Optionen `ACTIVE` und `INACTIVE`

unterscheidet. Diese Funktion wurde eingebaut, damit Nutzer:innen diese automatisierten Tasks pausieren können, ohne die entsprechenden Konfigurationseinträge löschen zu müssen. Das `timeframe`-Attribut gibt an, in welchen Abständen die Aufgaben angestoßen werden, beispielsweise wird ein Task mit dem Wert `1H` stündlich ausgeführt. Eine detaillierte Übersicht der möglichen Werte ist in Tabelle 4 zu finden.

Wert	Zeiteinheit
5-59M	Minuten
1-23H	Stunden
1-6D	Tage
1-4W	Wochen

Tabelle 4: Mögliche Werte der Klasse `Timeframe`

Ein weiteres wichtiges Attribut ist `start`, welches den Startzeitpunkt im Format `HH:mm` angibt. Sobald dieser definierte Zeitpunkt das nächste Mal erreicht wird, wird die Ausführung des Tasks erstmals initiiert. Die Ausführung wird erst gestoppt, wenn der Eintrag komplett gelöscht oder anhand des Status-Attributs deaktiviert wird. Bei einem Neustart des Systems werden alle Tasks wieder nach dem gleichen Prinzip gestartet, so dass die zeitliche Reihenfolge beibehalten wird.

Besonders hervorzuheben ist das Attribut `overwrite`, das die grundlegende Verwaltung von automatisch extrahierten Daten unterscheidet. Diese Option bestimmt, ob extrahierte Daten jedes Mal überschrieben (ON) oder neu gespeichert (OFF) werden sollen. Bei beiden Optionen werden die gesammelten Daten in Einträgen des Formats: `[AutomatedConfig.dbName]_[UUID.random]` gespeichert. Dadurch entstehen einzigartige Namen für die extrahierten Strukturen, welche der Konfigurationsdatenbank durch die ähnliche Benennung zugeordnet werden können. Dies ist entscheidend bei der Überschreibung der Daten: Jedes Mal, wenn der Vorgang durch den Scheduler gestartet wird, werden zunächst alle zuordbaren Daten gelöscht, sofern `overwrite` auf ON gestellt ist und die neuen Daten eingefügt.

Durch diese Implementierung erhalten Nutzer:innen die Möglichkeit, die Konfigurationsdaten flexibel anzupassen, wodurch eine detaillierte Steuerung des automatisierten Web-Scrapings gewährleistet wird. Um die Übersichtlichkeit und Wartbarkeit des Systems zu sichern, muss auf eine klare Trennung zwischen den tatsächlichen Daten und den Konfigurationsdaten geachtet werden.

3.2.4 Daten für Einstellungen

Neben den fachlichen Daten müssen auch die gesetzten Einstellungen persistent gespeichert werden. Dies erfolgt über das Objekt `SettingsData`, das den abschließenden Bestandteil des gesamten Datenmodells darstellt, vgl. Tabelle 5.

Attributname	Datentyp	Beschreibung
id	Long	Statische id
amountThreadsForScraping	Integer	Anzahl zugewiesener Threads
delayForScraping	Integer	Verzögerung vor Laden des Quellcodes
minRowsPerStructure	Integer	Minimale Zeilenanzahl je Struktur
maxRowsPerStructure	Integer	Maximale Zeilenanzahl je Struktur
amountExampleDataHome	Integer	Vorschaugröße unter /home
amountExampleDataManual	Integer	Vorschaugröße unter /manual
typeOfDataStructure	String	Selektion von Strukturen des Scraping

Tabelle 5: Attribute der Klasse `SettingsData`

Da lediglich eine Instanz für die Einstellungsdaten existiert, wurde sich bewusst gegen die Verwendung einer UUID entschieden. Stattdessen wird ein fester Long-Wert mit dem Wert 1 verwendet, um die Speicherung einfacher zu gestalten.

Die Anzahl der Threads für das Scraping bestimmt, in wie viele Bereiche die Daten eingeteilt werden, um anschließend parallel verarbeitet zu werden. Eine höhere Anzahl beeinflusst die Geschwindigkeit des Vorgangs positiv. Es wird außerdem geprüft, dass die Anzahl verfügbarer Threads nicht überschritten werden darf.

Für das Attribut `delayForScraping` wird eine Zeitspanne in Sekunden definiert, die der Web Scraper nach dem Aufruf der Seite abwartet, bevor die Extraktion startet. Dies ist insbesondere auf leistungsschwächeren Systemen wichtig, da der vollständige Seitenaufbau mehr Zeit in Anspruch nehmen kann. Ein größerer Wert bietet mehr Robustheit, führt allerdings gleichzeitig zu einer längeren Verarbeitungsdauer. Um Fehlkonfigurationen zu verhindern, wurde der Maximalwert auf 60 Sekunden beschränkt.

Die Attribute `minRowsPerStructure` und `maxRowsPerStructure` dienen zur Filterung unbedeutender Datenstrukturen. Dadurch lassen sich etwa sehr kleine Strukturen selektieren, die potenziell keine relevanten Ergebnisse enthalten.

Zur individuellen Anpassung der Darstellung können Nutzer:innen die Größe der Vorschaudaten beliebig wählen. Eine niedrige Anzahl ist dabei oft empfehlenswert, um die Übersichtlichkeit zu wahren.

Zuletzt legt das Attribut `typeOfDataStructure` fest, welche Strukturen erfasst werden sollen. Die Auswahl beschränkt sich auf Tabellen, Listen oder beide dieser Strukturen.

Zusammenfassend kann man festhalten, dass eine hohe individuelle Anpassungsmöglichkeit geboten wird, um auf persönliche Bedürfnisse eingehen zu können.

3.2.5 Gestaltungskriterien und Entwurfsziele

Das Design des Datenmodells und der Datenbankanbindung folgt mehreren zentralen Prinzipien, um eine robuste, flexible und wartbare Architektur gewährleisten zu können. Dabei besteht ein zentrales Ziel in der Trennung von Konfigurations- und Nutzdaten. Durch diese getrennte Speicherung bleibt das System übersichtlich und skalierbar. Anpassungen an den Scraping-Einstellungen haben keine Auswirkung auf die gespeicherten Inhalte.

Um eine hohe Wiederverwendbarkeit zu erreichen, wird für alle extrahierten Daten ein einheitliches Datenmodell genutzt. Unterschiede zwischen manuell und automatisiert gesammelten Daten werden dabei durch das `destination`-Attribut kenntlich gemacht.

Ein weiterer wichtiger Aspekt ist die Lesbarkeit und Nachvollziehbarkeit der Daten. Diese werden immer als strukturierte Liste von Schlüssel-Wert-Paaren gespeichert, was zur einheitlichen Verarbeitung im System beiträgt und eine enorm flexible Bearbeitung zulässt.

Bei der Gestaltung der Anwendung war die Minimierung von Redundanz ein wichtiges Kriterium. Metadaten, wie URL, Zeitstempel und Ursprung werden zentral verwaltet, sodass Mehrfachspeicherungen vermieden werden. Dadurch müssen Metadaten auch nicht in einer separaten Datenbankabfrage angefordert werden, sondern können zentral bezogen werden.

Alle Gestaltungsentscheidungen tragen dazu bei, die langfristige Wartbarkeit des Systems sicherzustellen und gleichzeitig die Übersicht und Trennschärfe der einzelnen Komponenten zu erhalten.

3.3 Frontend- und Backend-Integration

Die Integration von Frontend und Backend bildet die zentrale Komponente jeder Anwendung, da sie Benutzerinteraktion mit Geschäftslogik verbindet. Ziel des Frontends ist es, eine einfach zugängliche Benutzererfahrung zu ermöglichen. Es wurde darauf geachtet, dass das System auch Unterstützung für Bedienungshilfen bietet, indem ein geregelter Aufbau der Weboberfläche eingehalten wurde. Ein weiteres Ziel bestand in der Erfüllung von Accessibility-Eigenschaften, damit jeder die Anwendung uneingeschränkt nutzen kann.

Das Frontend besteht aus HTML-Dateien, welche die grundlegende Struktur der Anwendung festlegen und von Thymeleaf zur dynamischen Datenbindung genutzt werden. Das ermöglicht die Übermittlung von Werten, die durch Formulare übergeben werden und in der Geschäftslogik verarbeitet werden sollen. Um eine angenehme visuelle Erfahrung zu schaffen, wurden CSS-Dateien verwendet, welche das eigentliche Design realisieren. Bereits im Frontend müssen passende URLs aufgelöst werden, damit eine Weiterleitung zum Backend möglich ist, was in dem Fall auch dynamisch durch Thymeleaf realisiert wurde.

Das Backend wurde mit dem Spring Framework realisiert und stellt spezifische Endpunkte über Controller-Klassen bereit, welche die vorhandenen Routen auf ausführbaren Code abbilden. Diese Klassen sind alle mit der `@Controller`-Annotation versehen und

kommunizieren mit Service-Klassen, welche durch Springs Dependency Injection automatisch bereitgestellt werden. Die Service-Klassen kapseln Geschäftslogik und leiten Datenbankaufrufe an zuständige Repository-Komponenten weiter, um Abhängigkeiten des Anwendungskerns zu verhindern.

Insgesamt trägt die gewählte Umsetzung zu einer interaktiven und gleichzeitig robusten Anwendung bei. Durch die dedizierten Komponenten wird dem Model-View-Controller (MVC)-Prinzip (Contributors, 2025a) gefolgt und schafft die Möglichkeit einer unabhängigen Weiterentwicklung der einzelnen Stellen.

3.4 Sicherheitsaspekte

Bei der Entwicklung der Anwendung wurde der *Secure-by-Design*-Ansatz verfolgt, bei dem Sicherheitsaspekte bereits in der Entwurfsphase berücksichtigt wurden. Bei diesem Ansatz wird, insbesondere durch eine geeignete Architektur-Wahl, versucht, mögliche Fehlerquellen bereits im Vorhinein auszuschließen.

3.4.1 Schutz vor SQL-Injection

Relationale Datenbanken, die auf SQL basieren, sind sehr anfällig für sogenannte SQL-Injection-Angriffe. Bei dieser sehr verbreiteten Art von Angriffen versuchen Akteur:innen, ihren eigenen SQL-Code in die Anwendung einzuschleusen und auszuführen, um unautorisierte Aktionen auf der Datenbank durchzuführen (kingthorin und zbraiterman, 2025). Häufig werden dafür `<form>`-Attribute in HTML verwendet, welche ein Formular mit den übergebenen Daten an das Backend übermitteln, wo die Daten dann in Datenbankabfragen integriert werden. Um solche Angriffe zu vermeiden, sollte jede Datenbank-Abfrage im Backend sorgfältig geprüft und Eingaben durch sogenanntes Escaping validiert werden.

Durch den Einsatz von MongoDB als Datenbank entfällt diese Notwendigkeit, da MongoDB dokumentenbasiert arbeitet, also keine SQL-Syntax verwendet und somit auch keine Angriffsfläche für SQL-Injections bietet. Stattdessen finden alle Datenzugriffe über ein Repository-Interface statt, welches von Spring Data MongoDB bereitgestellt wird und mögliche Angriffsversuche automatisch wirkungsvoll unterbindet.

3.4.2 Validierung und Schutz vor unsachgemäßer Nutzung

Um sicherzustellen, dass Nutzer:innen keine Inkonsistenzen durch Fehleingaben verursachen, kommen umfangreiche Validierungen auf allen Eingabefeldern zum Einsatz. Diese werden serverseitig über Java Bean Validation und teilweise auch durch Datenbankabfragen realisiert. Ungültige Eingaben werden durch passende Fehlermeldungen im Frontend für Nutzer:innen visualisiert und verhindert. Ein gutes Beispiel hierfür findet sich bei der Konfiguration der Anzahl an Threads, die für das Web Scraping verwendet werden sollen unter `/Settings`. Hier wird strikt geprüft, dass nicht mehr Threads angegeben werden können, als zur Laufzeit verfügbar sind. Neben solchen Prüfungen werden auch weniger offensichtliche Aspekte berücksichtigt, die nur durch sehr tiefge-

hendes technisches Verständnis hervorgerufen werden können. Zum Beispiel werden in der Anwendung sogenannte `hidden input-fields` validiert, welche in der Benutzeroberfläche und in dem überlieferten Quellcode der HTML-Seite nicht sichtbar sind. Durch das gezielte Einfügen von eigenen Input-Feldern mit sehr spezifischer Benennung, könnten Akteur:innen hier potenziell Schäden anrichten, falls dies nicht im Backend unterbunden wird.

3.4.3 Zugriffssicherheit und zukünftige Erweiterung

Derzeit ist keine Authentifizierung oder Autorisierung innerhalb der Anwendung implementiert. Das liegt daran, dass die Anwendung für die lokale Nutzung ausgelegt ist und diese Funktionen daher nicht benötigt werden. Die Architektur wurde allerdings so konzipiert, dass eine solche Änderung in Zukunft problemlos durch `Spring Security` erweitert werden kann. Dadurch wäre es möglich, Benutzerrollen zu definieren, bestimmte Endpunkte abzusichern oder Login-Funktionen einzubauen, um ein zentrales System bereitzustellen, welches mehrere wissenschaftliche Mitarbeiter:innen zusammen nutzen können.

3.4.4 Fehlertoleranz und Schutz der Datenbankstruktur

Die Nutzung von MongoDB trägt zusätzlich zur Fehlertoleranz bei, indem keine strikten Schemata erzwungen werden, anders als bei relationalen Datenbanken. Der große Vorteil, den dieses Konzept bietet, ist die große Flexibilität und Freiheit bei der Speicherung. Alle Daten können in einer Datenbank gelagert und durch Metadaten abgegrenzt werden, was die Verwaltung erheblich erleichtert. Allerdings hat der Ansatz den Nachteil, dass man einen leicht erhöhten Validierungsaufwand im Backend hat. Um ein Beispiel zu nennen, kann die Speicherung von automatisch extrahierten Daten betrachtet werden: Bei der Speicherung muss u.a. geprüft werden, ob der angegebene Name nicht bereits in den gespeicherten Datensätzen vorkommt. Allerdings darf dieser in Datensätzen vorkommen, welche das Attribut `Destination` auf `Home` gesetzt haben. Bei einer relationalen Datenbank würde dies automatisch eine Fehlermeldung geben. Um auch eine korrekte inhaltliche Repräsentation der Daten sicherzustellen, werden Kontrollzeichen, wie z.B. Zeilenumbrüche oder Tabulator-Einrückungen aus den Daten entfernt und Optionen in der Bearbeitung von Daten bereitgestellt.

3.4.5 Fazit

Insgesamt bietet die Anwendung durch den Einsatz von Spring Data MongoDB, validierungs-basierten Schutzmechanismen und einer zugeschnittenen Architektur ein stabiles und sicheres Fundament. Obwohl auf eine vollständige Zugriffskontrolle inklusive Benutzerverwaltung verzichtet wurde, ist die Integration solcher Funktionen jederzeit problemlos möglich. Damit ist die Anwendung sowohl für lokale Nutzung als auch für eine potenzielle Weiterentwicklung ausgelegt.

4 Implementierung

In diesem Kapitel geht es um die konkrete Implementierung der zentralen Komponenten der Arbeit. Der Fokus liegt dabei auf der technischen Umsetzung des Web Scrapers, da dies das Kernkonzept dieser Arbeit darstellt. Des Weiteren werden die Frontend-Funktionen erläutert.

4.1 Implementierung des Web Scrapers

Zunächst wird erläutert, wie der Quellcode, der unter anderem auch die gesuchten Daten beinhaltet, aus der Webseite mithilfe von Selenium extrahiert wird und welche Vorkehrungen getroffen wurden, um dies zu ermöglichen. Danach wird auf die konkrete Extraktion der Daten aus diesem Quellcode eingegangen, wobei hier in Tabellen und Listen unterschieden wird. Da große Mengen an Daten fast ausschließlich in diesen HTML-Strukturen angezeigt werden, umfasst diese Arbeit auch nur diese. Zu beiden Punkten werden außerdem noch einige der möglichen Formen/Abwandlungen thematisiert, die der Scraping-Vorgang behandeln kann.

4.1.1 Quellcode-Extraktion durch Spoofing

Webseiten haben oft Interesse daran, ihre Daten vor automatischen Zugriffen, wie denen eines Web Scrapers, abzusichern. Dies kann durch verschiedene Mechanismen erzielt werden, die durch eine bewusste Manipulation umgangen werden können. Das nennt sich Spoofing und findet auch in dieser Arbeit Relevanz, indem unterschiedliche Techniken zur Tarnung des Web Scrapers angewendet werden. Es ist extrem wichtig zu erwähnen, dass diese Techniken gegen Richtlinien verstoßen können, weshalb der Einsatz nur unter ausdrücklicher Einverständniserklärung des Webseiten-Betreibers erfolgen darf.

Besonders häufig prüfen Webseiten Angaben des User-Agents, weshalb sich hier für die Angabe eines typischen Chrome-Browsers auf einem Windows-System entschieden wurde. Weitere wichtige Manipulationen erfolgen in den Navigator-Attributen, welche mit realistischen Werten befüllt wurden. Außerdem wurden zufällige Mausbewegungen und ein variables Scrollverhalten eingebaut. Die Mausbewegungen sollen menschliches Verhalten imitieren, während das Scrollverhalten das dynamische Laden von Daten erzwingt und durch Ausführung von JavaScript-Code umgesetzt wird.

Durch die Kombination der genannten und weiteren Techniken wird ein fehlertoleranter Zugriff auf Webseiten ermöglicht, der zur korrekten und vollständigen Extraktion beiträgt und somit die Grundlage des Scraping-Vorgangs schafft.

4.1.2 Extraktion aus Tabellen

Tabellen sind im Quellcode als `table`-Elemente (WHATWG, 2025) definiert und können anhand dessen vom Web Scraper gefunden werden. Bevor die Verarbeitung umgesetzt wird, werden verschachtelte Tabellenstrukturen entfernt und nur die äußerste Struktur betrachtet. Dieser Umgang mit verschachtelten Tabellen bietet eine einfache Lösung, bei der enorme Komplexität entfällt. Dieser Ansatz wurde gewählt, da verschachtelte Tabellen sehr selten vorkommen und somit die Kosten der Implementierung nicht durch eine sinnvolle Nutzung kompensiert werden. Wenn nun die Tabellen-Elemente gefunden wurden, werden diese einzeln der Reihe nach verarbeitet. Dafür wird geprüft, welche Form die Tabelle hat und die entsprechende Extraktionsmethode angewandt. Dies ist erforderlich, da Tabellen Abweichungen in ihrer Struktur aufweisen können, indem bestimmte Attribute, wie z.B. der Tabellenkopf (`thead`) optional sind und die Datenbeschaffung erschweren. Falls keine passenden Überschriften gefunden werden, generiert eine Methode diese automatisch. Außerdem werden identische Tabellennamen durchnummeriert, um eine eindeutige Zuordnung zu schaffen.

Beim Ausführen der zugehörigen Methode wird die Struktur dann passend abgearbeitet, wobei auch das Konzept Multithreading, siehe Abschnitt 4.1.4, zum Einsatz kommt. In jeder Methode müssen spezifische Besonderheiten beachtet werden, z.B. ein mehrzeiliger Tabellenkopf, vgl. Tabelle 6 oder das Fehlen eines Tabellenkopfes.

Kategorie	Produkte			Gesamt
	Elektronik		Haushalt	
	Mobil	PC		
Verkauf Januar	100	200	150	450
Verkauf Februar	120	180	170	470

Tabelle 6: Tabelle mit mehrzeiligem Tabellenkopf

Neben diesen Besonderheiten kann der Web Scraper vertikale Schlüssel-Wert-Tabellen passend auflösen. Diese Art von Tabellenstruktur wird häufig verwendet, um sehr kleine Datenmengen, wie Definitionen, personenbezogene Daten oder kleine Übersichten darzustellen. Da einem Wert hier manchmal mehrere Daten innerhalb der Webseite zugewiesen werden, die durch Trennsymbole optisch unterschieden werden, muss gegebenenfalls eine Nachbereitung der Daten stattfinden. Da diese aber spezifisch an die jeweiligen Trennsymbole angepasst sein muss, wurde dieser Aspekt hier nicht berücksichtigt.

Der häufigste Anwendungsfall besteht bei der Extraktion von vollständig strukturierten Tabellen. Diese enthalten sowohl einen `thead`- als auch einen `tbody`-Bereich. Die Überschriften und Wert werden in diesem Format strikt getrennt, sodass eine Sammlung der Daten vereinfacht wird. In dieser Form finden auch mehrzeilige Kopfzeilen, wie bereits dargestellt, Anwendung und werden mithilfe einer spezialisierten Methode vom System verarbeitet und in passende einzeilige Spaltennamen aufgeteilt.

Auch andere Formate, die keine oder nur eine partielle Struktur enthalten, werden ebenfalls vom Scraper berücksichtigt und erfasst. Darüber hinaus werden leere Tabellen oder

Tabellen mit leeren Zeichenketten nicht in das Endergebnis überführt, um keine unnötigen Daten zu speichern.

Falls keine passende Methode gefunden wird, verwendet die Anwendung eine sehr allgemeine Methode und versucht dennoch, die Daten so gut wie möglich zu sammeln. Diese Methode dient auch als Fallback-Methode, wird also verwendet, wenn die eigentlich zugehörige Methode unerwartet fehlschlägt. Auftretende Fehler bei der Extraktion beeinflussen das System nicht, sondern resultieren im schlimmsten Fall zur Nicht-Erfassung von einzelnen Tabellen-Elementen.

4.1.3 Extraktion aus Listen

Neben der Datenextraktion aus Tabellen stellt die Sammlung von Daten aus Listen ein weiteres zentrales Ziel der Anwendung dar. Listen kommen sehr oft auf Webseiten vor und können dabei verschiedene Formen annehmen. Im Gegensatz zu Tabellen ist die Variation zwar geringer, allerdings werden Listen häufiger verschachtelt verwendet oder enthalten ungeordnete Daten, was die Extraktion erheblich erschweren kann. Das entwickelte System kann solche Anwendungsfälle erkennen und bietet eine entsprechende Lösung.

Der Web Scraper prüft zunächst auf Verschachtelungen in Listen (`ul`- und `ol`-Elemente) und verarbeitet diese. Die Verarbeitung erfolgt, indem die jeweiligen Ebenen der Verschachtelung als einzelne Spalten dargestellt werden. Durch diese Methodik wird eine klare und übersichtliche Form beibehalten, was essentiell ist bei dieser Komplexität.

Neben verschachtelten Listen werden auch strukturierte Listen erfolgreich behandelt. Dabei sind mehrere HTML-Elemente innerhalb der einzelnen Einträge vorhanden und werden korrekt in den jeweiligen Textinhalt aufgelöst. Die verschiedenen HTML-Elemente werden außerdem in einzelne Spalten aufgeteilt. Vorhandene Zeilenumbrüche weisen auf eine zusätzliche Unterteilung von Daten hin, weshalb in diesem Fall eine weitere Aufteilung erfolgt.

Neben diesen eher komplexeren Formen werden auch ganz einfache Listen behandelt. Nur bei dieser Form wird Multithreading verwendet, da hier oft größere Datenmengen gespeichert werden und die Implementierung innerhalb der anderen Listenstrukturen in einem ungerechtfertigten Mehraufwand resultieren würde.

Insgesamt wurde außerdem darauf geachtet, dass keine unstrukturierten Daten bei dem Prozess entstehen. Falls also Spalten gar keine Werte haben, werden diese automatisch mit Whitespaces (leeren Werten) befüllt, damit eine konsistente Struktur erhalten bleibt. Dies ist besonders wichtig, um keinen zusätzlichen Bearbeitungsaufwand zu verursachen, wenn diese Daten in andere Formate exportiert und dort verarbeitet werden. Außerdem wurde explizit geprüft, dass leere Listen oder reine Whitespace-Werte keinen Einfluss auf das Endergebnis haben.

4.1.4 Parallele Verarbeitung durch Multithreading

Ein zentrales Ziel der Anwendung besteht auch darin, eine möglichst effiziente Lösung beim Scraping von großen Datenmengen bereitzustellen. Insbesondere ist diese Möglichkeit für das automatische Scrapen wichtig, da hier oft in relativ kleinen Zeitintervallen viele Daten abgefragt werden können.

Da Threads einen sogenannten Overhead (Jenkov, 2021) verursachen, also Performanz durch den Verwaltungsaufwand dieser einbüßen, muss auf eine passende Implementierung Wert gelegt werden. Es würde beispielsweise keinen Sinn ergeben, die einzelnen Zeilen einer Tabelle in jeweils einem Thread zu bearbeiten, da der Rechenaufwand einer einzelnen Zeile im Vergleich zum Overhead zu gering ausfällt. Daher wurde eine besondere Methode implementiert, die mehrere Zeilen in gleich große Bereiche, anhand der zugewiesenen Thread-Anzahl, aufteilt. Die Teilergebnisse werden dann synchron gesammelt und dem Gesamtergebnis in passender Reihenfolge beigefügt.

Aufgrund dieses Ansatzes können erhebliche Leistungsunterschiede bei der Verarbeitungsgeschwindigkeit festgestellt werden, ohne dabei die Logik der Zeilenverarbeitung selbst anpassen zu müssen. Dadurch können Änderungen einfach umgesetzt und neue Mechanismen eingebaut werden.

4.2 Frontend-Funktionen und Benutzeroberfläche

Die Webanwendung stellt eine moderne und interaktive Benutzeroberfläche zur Verfügung, die zahlreiche Funktionen für Nutzer:innen zugänglich macht. In diesem Kapitel werden diese genauer aufgeführt, wobei der Fokus auf der Exportmöglichkeit in CSV- und JSON-Formate liegt.

Zunächst können Nutzer:innen einen manuellen Scraping-Vorgang starten, indem sie eine Ziel-URL unter `/manual` bereitstellen und den Prozess starten. Anschließend wird eine Übersicht an gefundenen Tabellen- und Listenstrukturen angezeigt, die neben der jeweiligen Größe der gefundenen Elemente auch einen kleinen Ausschnitt der Daten beinhalten. Dadurch wird neben einer besseren Performanz, da nicht alle Daten ans Frontend geschickt werden, auch eine übersichtliche Struktur geschaffen, die zur Benutzerfreundlichkeit beiträgt. Falls man dennoch mehr Daten in der Übersicht angezeigt bekommen möchte, kann dies unter `/settings`, neben anderen grundlegenden Konfigurationen, eingestellt werden. Anschließend können Benutzer:innen alle benötigten extrahierten Tabellen- und Listen auswählen und unter einem anpassbaren Namen speichern.

Eine andere Weise die Anwendung zu nutzen, besteht darin automatisierte Scraping-Aufträge zu bestimmen. Dafür können Nutzer:innen unter `/automated` ihre bereits erstellten Aufträge sehen, diese bearbeiten oder neue hinzufügen, vgl. Tabelle 7.

status	url	timeframe	start	database-name	overwrite	
Active	https://example1.com	1D	19:07	<u>testName1</u>	On	<u>Edit</u>
Inactive	https://example2.com	3H	08:00	<u>testName2</u>	Off	<u>Edit</u>

Tabelle 7: Übersicht der Aufträge unter `/automated`

Zusätzlich kann man hier unter der Spalte *database-name* zu einer weiteren Übersicht gelangen, in der alle automatisch extrahierten Strukturen des Auftrages aufgelistet werden, vgl. Tabelle 8.

database-name	scraped at	#rows	url	
testName1_eed4d9dd-...	2025-07-13T19:07:32.863	196	https://example1.com	<u>Save</u>
testName1_0d5c24a0-...	2025-07-13T19:07:34.237	217	https://example2.com	<u>Save</u>
testName1_affe479-...	2025-07-13T19:07:41.144	5	https://example2.com	<u>Save</u>

Tabelle 8: Übersicht der Aufträge unter `/automated/testName1`

Dort kann man die Daten nach `/home` kopieren, um weitere Bearbeitungen oder einen Export vorzunehmen. Dadurch sichert man die Daten, falls die Überschreibung aktiviert wurde. Außerdem kann man sich die exakt extrahierten Daten in voller Größe anschauen, wenn man den Link unter *db_name* aufruft, vgl. Tabelle 9.

Counry/Economy	GDP (billions of \$): 2021	Share (%)	Rank	Continent
United States	23,315.075	25.4	1	North America
China	17,759.307	17.9	2	Asia
...	...	...	...	...

Tabelle 9: Übersicht spezifischer Daten, verändert entnommen aus:

<https://statisticstimes.com/economy/countries-by-gdp.php>

Der bereits erwähnte Endpunkt `/home` dient hauptsächlich zur Verwaltung gesammelter Ergebnisse. Bei der Navigation auf eine spezifische Tabelle oder Liste, kann man dort einzelne Spalten dieser löschen oder umbenennen. Um diesen Prozess zu erleichtern, werden Zwischenstände angezeigt, die man im Nachhinein zur Übernahme der Änderungen bestätigen muss. Des Weiteren können Nutzer:innen ganze Strukturen löschen oder in CSV- sowie JSON-Formate exportieren, wie im Folgenden zu sehen.

```
United States,"23,315.075","25.4","1",North America
China,"17,759.307","17.9","2",Asia
...
```

Listing 2: Daten im CSV-Format (ohne Typzuweisung)

```
[ {
  "Country/Economy" : "United States",
  "GDP (billions of $): 2021" : "23,315.075",
  "Share (%)" : "25.4",
  "Rank" : "1",
  "Continent" : "North America"
}, {
  "Country/Economy" : "China",
  "GDP (billions of $): 2021" : "17,759.307",
  "Share (%)" : "17.9",
  "Rank" : "2",
  "Continent" : "Asia"
}, {
  ...
} ]
```

Listing 3: Daten im JSON-Format (ohne Typzuweisung)

Bei dem Export lässt sich zusätzlich einstellen, ob versucht werden soll Werte auf spezifische Datentypen zu übertragen. Technisch werden sie dabei in die drei aufgelisteten Möglichkeiten unterteilt, wie aus Tabelle 10 entnommen werden kann.

Datentyp	Beispiel (als String exportiert)	Beispiel (nach Typ-Zuweisung)
Ganzzahl (Integer)	"42"	42
Dezimalzahl (Double)	"841,419.45"	841419.45
Boolean	"true"	true
Datum	"15.08.2024"	2024-08-15
Datum/Zeit	"2024-08-15T09:30:00"	2024-08-15T09:30:00
Text (String)	"Beispieltext"	"Beispieltext"

Tabelle 10: Beispiele für die automatische Typ-Zuweisung beim Export

Alle Interaktionen beinhalten eine Validierung mit entsprechender Rückmeldung an die Nutzer:innen. Die Benutzeroberfläche wurde intuitiv gestaltet, sodass keine zusätzliche Anleitung erforderlich ist.

5 Evaluation

Im Rahmen der Entwicklung des Web Scraping-Systems wurden umfangreiche Tests entwickelt, um die korrekte Funktionsweise sicherzustellen. In diesem Abschnitt werden die verschiedenen Komponenten evaluiert und explizite Szenarien vorgestellt, die Relevanz haben.

5.1 Controller- und Datenbank Tests

Für jeden Controller der vier bereitgestellten Endpunkte: `/home`, `/manual`, `/automated` und `/settings` wurden alle implementierten Methoden getestet. Zur Umsetzung dieser Tests kamen die Bibliotheken JUnit, AssertJ und Mockito zum Einsatz. Darüber hinaus wurde der von Spring bereitgestellte `WebMvcTest` verwendet.

Da Get-Requests keine Änderungen am Zustand der Anwendung vornehmen sollten (Fielding und Reschke, 2014), fokussieren sich die zugehörigen Tests auf grundlegende Aspekte. Hier wird u.a. sichergestellt, dass der korrekte Status-Code gesetzt wird, die richtige View zurückgegeben wird und bestimmte Attribute, gegebenenfalls mit spezifischen Werten, im Model vorhanden sind, vgl. Listing 4.

```
@Test
@DisplayName("GET_/automated_returns_correct_overview")
void testAutomatedOverview() throws Exception {
    when(automatedConfigService.findAll()).thenReturn(testConfigs);

    mockMvc.perform(get("/automated"))
        .andExpect(status().isOk())
        .andExpect(view().name("/automated/overview"))
        .andExpect(model().attributeExists("entries"))
        .andExpect(model().attribute("entries", hasSize(1)));

    verify(automatedConfigService).findAll();
}
```

Listing 4: Typisches Testszenario bei Get-Requests

Post-Requests werden zwar auf die gleiche Art implementiert, sind allerdings oft umfassender gestaltet, da hier die Prüfung komplexer Geschäftslogik im Vordergrund steht. Anders als bei Get-Requests haben sie keine Garantien und können deswegen jegliche Zustandsänderungen umsetzen, die durch die Tests abgedeckt werden müssen.

Bei den Datenbanktests wird `DataMongoTest` von Spring verwendet, wobei der Fokus auf der Interaktion mit MongoDB liegt. Die einzelnen Testfälle stellen sicher, dass neben grundlegenden CRUD-Abfragen auch komplexere Abfragen das gewünschte Ergebnis liefern und dabei keine Inkonsistenzen verursachen.

5.2 Scraping von Tabellen

Für die technische Umsetzung der Prüfung wurden Unit-Tests implementiert, wobei die Anwendungsfälle in Form von HTML-Syntax direkt als `String` übergeben werden. Das

ermöglicht eine einfache und schnelle Möglichkeit zum Testing, ohne dass Quellcode spezifischer Webseiten zugezogen werden muss.

Zur Bewertung der Extraktionslogik von Tabellen wurde das Verhalten des Systems mit verschiedenen HTML-Tabellenformaten getestet. Dabei wurde insbesondere überprüft, ob die Struktur korrekt eingeordnet wurde, passende Überschriften generiert und die Daten vollständig erfasst wurden. Diese Testfälle decken die wesentlichen im System integrierten Formate ab, die in dem Kapitel der Implementierung genannt wurden.

Im Folgenden werden diese Fälle aufgelistet und Abbildungen bereitgestellt, welche den entsprechenden Quellcode und die zugehörige Darstellung zeigen. Die Extraktion wurde in allen dieser Fälle erfolgreich durchgeführt und bietet daher eine gute Übersicht der behandelten Aspekte.

- **Vertikale Key-Value-Tabellen**
vgl. Listing 5 und Tabelle 11
- **Vollständig strukturierte Tabellen mit `thead` und `tbody`**
Hier explizit mit mehrzeiligen Überschriften, vgl. Listing 6, Tabelle 12 und Tabelle 13
- **Tabellen ohne `thead`, aber mit `th` in der ersten Zeilen**
vgl. Listing 7 und Tabelle 14
- **Tabellen ohne `thead` und ausschließlich mit `td`-Elementen**
vgl. Listing 8 und Tabelle 14
- **Komplexe bzw. unbekannte Tabellenstrukturen (Fallback)**
vgl. Listing 9 und Tabelle 15

```
<table>
  <tr><th>Full Name</th><td>Alice Johnson</td></tr>
  <tr><th>Age</th><td>28</td></tr>
  <tr><th>Occupation</th><td>Software Engineer</td></tr>
  <tr><th>Location</th><td>San Francisco</td></tr>
  <tr><th>Email</th><td>alice@example.com</td></tr>
</table>
```

Listing 5: Zugehörige Tabelle als HTML-Code

Full Name	Alice Johnson
Age	28
Occupation	Software Engineer
Location	San Francisco
Email	alice@example.com

Tabelle 11: Vertikale Key-Value-Tabelle

```

<table>
  <thead>
    <tr>
      <th rowspan="2">Experiment</th>
      <th colspan="2">Messwerte</th>
      <th colspan="2">Abweichung</th>
    </tr>
    <tr>
      <th>Sensor A</th>
      <th>Sensor B</th>
      <th>Absolut</th>
      <th>Relativ (%)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Test1</td><td>23.4</td><td>22.8</td><td>0.6</td><td>2.56%</td>
    </tr>
    <tr>
      <td>Test2</td><td>21.1</td><td>20.9</td><td>0.2</td><td>0.95%</td>
    </tr>
    <tr>
      <td>Test3</td><td>25.0</td><td>24.1</td><td>0.9</td><td>3.60%</td>
    </tr>
  </tbody>
</table>

```

Listing 6: HTML-Code einer vollständig strukturierten Tabelle mit mehrzeiligem Kopf

Experiment	Messwerte		Abweichung	
	Sensor A	Sensor B	Absolut	Relativ (%)
Test 1	23.4	22.8	0.6	2.56%
Test 2	21.1	20.9	0.2	0.95%
Test 3	25.0	24.1	0.9	3.60%

Tabelle 12: Zugehörige Tabelle im Originalformat

Experiment	Messwerte: Sensor A	Messwerte: Sensor B	Abweichung: Absolut	Abweichung: Relativ (%)
Test 1	23.4	22.8	0.6	2.56%
Test 2	21.1	20.9	0.2	0.95%
Test 3	25.0	24.1	0.9	3.60%

Tabelle 13: Tabelle nach erfolgreicher Extraktion durch den Web Scraper


```
<table>
  <tr><th>City</th><th>Population</th><th>Country</th></tr>
  <tr><td>Paris</td><td>2161000</td><td>France</td></tr>
  <tr><td>London</td><td>8982000</td><td>UK</td></tr>
  <tr><td>Berlin</td><td>3669000</td><td>Germany</td></tr>
</table>
```

Listing 7: HTML-Code einer Tabelle ohne `thead` und `tbody`, aber mit `th`-Elementen

City	Population	Country
Paris	2161000	France
London	8982000	UK
Berlin	3669000	Germany

Tabelle 14: Zugehörige Visualisierung der Tabelle

```
<table>
  <tr><td>City</td><td>Population</td><td>Country</td></tr>
  <tr><td>Paris</td><td>2161000</td><td>France</td></tr>
  <tr><td>London</td><td>8982000</td><td>UK</td></tr>
  <tr><td>Berlin</td><td>3669000</td><td>Germany</td></tr>
</table>
```

Listing 8: Gleicher HTML-Code, aber ohne `th`-Elemente zur Definition der Kopfzeile

```

<table>
  <tr><td>Framework</td><th>Type</th><td>Language</td></tr>
  <tr><td>React</td><td>Frontend</td><td>JavaScript</td></tr>
  <tr><td>Django</td><td>Backend</td><td>Python</td></tr>
</table>

```

Listing 9: HTML-Code einer komplexeren Tabelle mit gemischten `td`- und `th`-Elementen

Framework	Type	Language
React	Frontend	JavaScript
Django	Backend	Python

Tabelle 15: Visualisierung einer solchen Tabelle

Fazit

Neben den gezeigten Szenarien werden noch weitere Variationen berücksichtigt, die u.a. Mischungen der einzelnen vorgestellten Fälle abdecken. Die wichtigsten Fälle wurden allerdings bereits hier aufgeführt.

Abschließend lässt sich über die Bewertung der Tabellenextraktion sagen, dass das entwickelte System eine Unterstützung für viele unterschiedlich strukturierte Tabellen bietet. Aufgrund der implementierten Strategien zur Erkennung verschiedener Formate – von vollständigen Tabellen mit mehrzeiligem Kopfbereich bis hin zu generischen Tabellen – konnte eine breite Abdeckung realisiert werden. Dabei erfolgt das Scraping zuverlässig und fehlertolerant, selbst bei uneinheitlicher oder außergewöhnlicher Struktur. Einschränkungen bestehen lediglich bei verschachtelten Tabellen, die aus Komplexitätsgründen bewusst ausgeschlossen wurden. Insgesamt ist das System in der Lage, eine Vielzahl realer Anwendungsfälle erfolgreich zu bewältigen.

5.2.1 Scraping von Listen

In diesem Abschnitt werden die jeweiligen Variationen von Listen visuell dargestellt, die vom System verwertet werden können. Diese wurden bereits in dem jeweiligen Abschnitt der Implementierung erläutert. Die Prüfung der korrekten Verarbeitung erfolgt auch hier über Unit-Tests, äquivalent zu der Vorgehensweise bei dem Scraping von Tabellen. Die folgenden Fälle zeigen typische Ausprägungen von Listen.

- **Tief verschachtelte Listen**
vgl. Listing 10 und Tabelle 16
- **Strukturierte Listenelemente, welche mehrere HTML-Elemente enthalten**
vgl. Listing 11 und Tabelle 17
- **Einfache, flache Listen (Beispiel hier whitespace-dominant)**
vgl. Listing 12 und Tabelle 18

```
<ul>
  <li>Main Topic
    <ul>
      <li>Subtopic 1
        <ul>
          <li>Detail A</li>
          <li>Detail B</li>
        </ul>
      </li>
      <li>Subtopic 2</li>
    </ul>
  </li>
  <li>Second Topic</li>
  <li>Third Topic</li>
</ul>
```

Listing 10: Liste mit tiefgehender Verschachtelung

Column_1	Column_2	Column_3
Main Topic	Subtopic 1	Detail A
Main Topic	Subtopic 1	Detail B
Main Topic	Subtopic 2	" "
Second Topic	" "	" "
Third Topic	" "	" "

Tabelle 16: Visualisierung der zugehörigen Daten nach Extraktion

```

<li>
  <h1>Article Title</h1>
  <p>Article description goes here</p>
  <div>Category: Technology</div>
  <a href="/article/1">Read more</a>
</li>

```

Listing 11: Ausschnitt eines Listeneintrags mit mehreren HTML-Elementen

Column_1	Column_2	Column_3	Column_4
Article Title	Article description goes here	Category: Technology	Read more

Tabelle 17: Passende Aufteilung in Spalten des zugehörigen Codes

```

<ul>
  <li> Item with spaces </li>
  <li></li>
  <li>

  </li>
  <li>Valid item</li>
</ul>

```

Listing 12: Einfache Liste mit leeren oder whitespace-dominierten Einträgen

Column_1
Item with spaces
Valid item

Tabelle 18: Visualisierung der zugehörigen einfachen Liste

Fazit

Abschließend lässt sich feststellen, dass das System in der Lage ist, verschiedene Formate an Listen zuverlässig zu erkennen und in eine geeignete Form zu übertragen. Sowohl einfache, strukturierte als auch tief verschachtelte Listen werden dabei zuverlässig vom Web Scraper extrahiert. Durch automatische Behandlung leerer Felder und einer sauberen Unterteilung wird eine hohe Qualität dieser gesammelten Daten gewährleistet.

5.3 Einschränkungen des Systems

Trotz zahlreicher positiver Resultate und der robusten Funktionsweise gibt es auch einige Einschränkungen des entwickelten Web Scraping-Systems, die nun betrachtet werden. Es gibt sehr fortgeschrittene Sicherheitsvorkehrungen, die eine Extraktion mit dem entwickelten System verhindern können. Des Weiteren kann nicht garantiert werden, dass alle Tabellen- und Listenstrukturen erfolgreich durchsucht werden, falls es sich um sehr spezifische Abwandlungen handelt. Falls Daten in unstrukturierter Form vorliegen und nicht in vorgesehenen Strukturen, werden diese vom System auch nicht erkannt.

Außerdem werden verschachtelte Tabellen nicht rekursiv verarbeitet, sondern es wird ausschließlich die äußerste Ebene berücksichtigt. Auch Kombinationen zwischen Tabellen- und Listenelementen, wie beispielsweise Listen innerhalb von Tabellenzeilen, werden derzeit nicht vollständig extrahiert. Darüber hinaus werden Definitionslisten aktuell nicht erkannt.

Ein weiterer Aspekt bezieht sich auf die Handhabung von extrem großen Datenmengen. Die Anwendung ist für den Umgang mit mittelgroßen bis großen Datensätzen optimiert, stößt allerdings an seine Grenzen, falls es darüber hinausgeht. Solche Grenzen machen sich durch hohe Verarbeitungszeiten bemerkbar, die die Nutzung und die Benutzererfahrung erheblich beeinträchtigen können. Allerdings variieren die genauen Einschränkungen aufgrund von leistungsfähigen Unterschieden der Technologien und können durch eine Lastverteilung behoben werden. Falls man die Anwendung für den Big-Data-Bereich ausbauen möchte, benötigt man zudem eine entsprechende Datenbank, um eine reibungslose Funktionsweise zu gewährleisten.

Technisch ist die Fehlerverwaltung nicht optimal implementiert, da keine eigenen internen Fehlermeldungen zum Einsatz kommen und von sogenannten Exception-Handleern automatisch verarbeitet werden. Bei ungewöhnlichen Fehlerquellen wird dadurch die Diagnostik und Behebung dieser erschwert.

Zusammengefasst bietet das System eine solide und umfangreiche Grundlage für die Extraktion von Daten aus HTML-Quellcode, jedoch ist sowohl die Leistung bei sehr großen Datenmengen als auch die Flexibilität bei stark unstrukturierten Webseiten-Aufbauten optimierbar. Künftige Verbesserungen könnten diese Einschränkungen verringern und zu einer verbesserten Skalierbarkeit sowie Benutzerfreundlichkeit beitragen.

6 Diskussion

In diesem Abschnitt der Arbeit werden sowohl die Stärken als auch mögliche Einschränkungen des Systems diskutiert. Basierend auf den gewonnenen Erfahrungen werden weiterführende Perspektiven und Entwicklungsmöglichkeiten vorgestellt, die in zukünftigen Versionen der Anwendung umgesetzt werden können.

6.1 Erfüllung der Anforderungen

Im Rahmen dieser Bachelorarbeit wurden verschiedene Anforderungen definiert (vgl. Kapitel 2), die das entwickelte System erfüllen sollte, um eine zuverlässige und korrekte Lösung für die automatisierte Datenextraktion von Webseiten bereitzustellen. Die genannten Anforderungen wurden dabei in verschiedene Aspekte unterteilt, die hier genauer analysiert werden.

6.1.1 Funktionale Anforderungen

Die wesentlichen funktionalen Anforderungen bestehen dabei aus der korrekten Entnahme von Daten aus Tabellen- und Listenstrukturen verschiedenster Formen. Diese Anforderungen werden vom System umgesetzt, indem spezialisierte Extraktionsmethoden entworfen wurden, die an die unterschiedlichen Formen der Strukturen angepasst sind. Die Vielzahl der abgedeckten Möglichkeiten trägt maßgeblich zur Flexibilität und Robustheit bei. Dazu trägt auch die Implementierung von Fallback-Methoden (Ausweich-Methoden) bei, wobei mögliche Fehler direkt behandelt und Alternativen geboten werden.

Ein weiteres Ziel bestand in der Möglichkeit, extrahierte Daten in einer Datenbank zu speichern und die Interaktion der Anwendung über eine grafische Oberfläche zu ermöglichen. Auch diese Anforderungen konnten erfolgreich erfüllt werden, insbesondere durch Nutzung der NoSQL-Datenbank MongoDB, welche besonders für solche Anwendungszwecke geeignet ist. Auch die Bereitstellung der Benutzeroberfläche wurde umfangreich umgesetzt und mit vielen Einstellungs- sowie Validierungsmöglichkeiten versehen, um das Arbeiten zugänglicher zu gestalten.

Zudem sollte die Möglichkeit geboten werden, extrahierte Daten in einer Datenbank zu speichern, sodass sie für die spätere Analyse und Bearbeitung zugänglich sind. Dies wurde durch die Integration von MongoDB als NoSQL-Datenbank erfolgreich umgesetzt. Die Nutzung von MongoDB ermöglicht eine flexible Speicherung der Daten ohne die Notwendigkeit einer starren Tabellenstruktur, was besonders gut zu den variierenden und teils unstrukturierten Daten passt, die durch den Web-Scraper extrahiert werden.

Außerdem sollte der Export von Daten in gängige Formate wie CSV oder JSON ermöglicht werden. Auch dieses Ziel wurde erfolgreich implementiert und beinhaltet sogar die Möglichkeit einer automatischen Typ-Zuweisung. Dadurch wird die weitere Verarbeitung und Auswertung der gesammelten Daten in anderen Informationssystemen erleichtert.

Weiterhin erlaubt das System, konfigurierbare Aufgaben (Tasks) für den automatischen

Scraping-Prozess zu definieren. Dies erlaubt eine Datenerfassung über gewisse Zeiträume und findet daher eine Vielzahl an Einsatzmöglichkeiten in der Forschung und darüber hinaus.

6.1.2 Nicht-funktionale Anforderungen

Neben den funktionalen Aspekten wird das System im Folgenden auch anhand angestrebter Qualitätsmerkmale bewertet. Als Ziele wurden insbesondere Performanz, Wartbarkeit und Skalierbarkeit definiert.

Die effiziente und schnelle Verarbeitung wird durch Anwendung von Multithreading erreicht, wobei die rechenintensive Verarbeitung parallelisiert wird. Die Daten wurden gezielt in gleich große Blöcke unterteilt und jeweils einzeln abgearbeitet. Insbesondere bei großen Datenmengen und leistungsstarken Prozessoren kann dies erhebliche Unterschiede ausmachen.

Die Wartbarkeit konnte durch den Einsatz bewährter Designmuster und modularer Programmierung gewährleistet werden. Es wurde darauf geachtet, eine saubere Trennung zwischen Komponenten beizubehalten, wodurch eine hohe Kohäsion und geringe Kopplung (Miller, 2019) entstanden sind. Dazu wurden insbesondere öffentliche Schnittstellen angeboten, durch welche die verschiedenen Komponenten miteinander interagieren können. Jede Funktionalität wurde in einer eigenen Komponente implementiert, um die Verantwortlichkeiten sauber zu differenzieren. Außerdem wurden bewährte Stile guter Programmierung angewandt, wie u.a. die SOLID-Prinzipien (Choe, 2020). Änderungen an einzelnen Komponenten sollten somit keine großen Auswirkungen auf unterschiedliche Bereiche der Anwendung haben, was zum weiteren Verlauf des Entwicklungsprozesses beiträgt. Einhergehend damit wird eine gute Skalierbarkeit sichergestellt, was vor allem auf die Wahl der Technologien, vor allem Spring, zurückzuführen ist.

6.1.3 Zusammenfassung

Zusammenfassend lässt sich sagen, dass die definierten Anforderungen, sowohl funktional als auch nicht-funktional, erfüllt wurden. Darüber hinaus wurden weitere Aspekte umgesetzt, die außerhalb der definierten Anforderungen liegen. Insgesamt wird eine flexible, leistungsstarke und wartbare Lösung zur automatisierten Datenerfassung auf Webseiten entwickelt. Auch in Zukunft sollte das System die beschriebenen Anforderungen weiterhin ordnungsgemäß erfüllen, sofern sich die Struktur von HTML nicht grundsätzlich ändert. Es bietet dadurch eine ideale Grundlage für Erweiterungen.

6.2 Weiterentwicklungsmöglichkeiten

Eine potentielle Weiterentwicklung des Systems besteht in der Umstrukturierung zu einer Multi-User-Plattform, die durch Spring Security abgesichert werden kann. Besonders in Bezug auf die Forschung könnte dies von großem Nutzen sein, da wissenschaftliche Mitarbeiter:innen auf zentrale Daten zugreifen können. Dort könnte man die Daten dann in private und öffentliche Rubriken einteilen, wodurch ein Austausch ermöglicht wird und dennoch die Privatsphäre geschützt bleibt.

Zusätzlich dazu könnte man das Benutzerinterface weiter verfeinern, was durch Bereitstellung von weiteren Anpassungsoptionen und Funktionen erfolgen kann. Beispielsweise könnte man einzelne Zeilen oder Werte bearbeiten, leere oder unvollständige Werte ersetzen lassen und Daten in weitere Spalten anhand von regulären Ausdrücken unterteilen.

Ein weiterer denkbarer Ausbau könnte die Integration von künstlicher Intelligenz sein, insbesondere als weitere Unterstützung bei der Verarbeitung hoch komplexer, fehlerhafter oder unvollständiger Datenstrukturen. Eine solche Erweiterung würde allerdings vergleichsweise hohe Kosten mit sich bringen, da die Nutzung von KI ressourcenintensiv und rechenaufwendig ist. Bei Verwendung einer bestehenden API müssten außerdem Abfrage-Limits und Nutzungskosten berücksichtigt werden.

Falls man auch Daten außerhalb von vorgesehenen Strukturen sammeln möchte, kann man mit komplexen Heuristiken arbeiten. Allerdings ist eine solche Implementierung sehr aufwändig und findet wahrscheinlich keinen großen Nutzen, aufgrund des geringen Vorkommens in der Praxis, weshalb es in dieser Arbeit nicht berücksichtigt wurde.

6.3 Fazit

Im Rahmen dieser Bachelorarbeit wurde ein flexibles und erweiterbares Web Scraping-System zur Extraktion strukturierter Daten aus Webseiten konzipiert und implementiert. Die entwickelte Anwendung ermöglicht dabei eine automatische Erkennung, Extraktion und Speicherung von Daten aus Tabellen und Listen, wobei verschiedene Variationen behandelt werden.

Durch die implementierte Architektur sowie die Nutzung aktueller Technologien wie Spring Boot, Selenium und Docker ist das System gut wartbar, plattformunabhängig und erlaubt individuelle Konfigurationen, die über eine benutzerfreundliche Oberfläche zur Verfügung gestellt werden. Eine besondere Stärke bietet dabei die Kombination zwischen paralleler Verarbeitung und berücksichtigter Formatvielfalt, wodurch auch größere Datenmengen effizient gesammelt werden können.

Trotz einzelner Einschränkungen stellt die entwickelte Lösung eine solide technische Basis dar, die sich bei Bedarf erweitern lässt und somit einen positiven Beitrag zur Forschungsarbeit leisten kann.

Literatur

- Johannes Boegershausen, Hannes Datta, Abhishek Borah und Andrew T. Stephen (Sep. 2022). *Fields of Gold: Scraping Web Data for Marketing Insights*. Department of Marketing Management. URL: <https://pure.eur.nl/en/publications/fields-of-gold-scraping-web-data-for-marketing-insights> (besucht am 28.07.2025).
- Yoonsuck Choe (2020). *SOLID Principles for Object-Oriented Design*. Folien vermutlich aus 2020. Texas A&M University. (Besucht am 28.07.2025).
- Spring Framework Contributors (2025a). 17. *Web MVC framework*. Jahr der Quelle nicht angegeben; Inhalt ist jedoch aktuell. Spring. URL: <https://docs.spring.io/spring-framework/docs/3.2.x/spring-framework-reference/html/mvc.html> (besucht am 28.07.2025).
- Spring Framework Contributors (2025b). *Dependency Injection*. Jahr der Quelle nicht angegeben; Inhalt ist jedoch aktuell. Spring. URL: <https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html> (besucht am 28.07.2025).
- Spring Framework Contributors (2025c). *Spring Framework Reference Documentation*. Jahr der Quelle nicht angegeben; Inhalt ist jedoch aktuell. Spring. URL: <https://docs.spring.io/spring-framework/reference/> (besucht am 28.07.2025).
- Docker (2025). *What is Docker?* Jahr der Quelle nicht angegeben; Inhalt ist jedoch aktuell. Docker, Inc. URL: <https://docs.docker.com/get-started/docker-overview/> (besucht am 28.07.2025).
- Ramesh Fadatare (Aug. 2024). *Thymeleaf vs React JS*. Jahr und Monat der Quelle nicht explizit angegeben; Lässt sich aus der URL erschließen. Java Guides. URL: <https://www.javaguides.net/2024/08/thymeleaf-vs-react-js.html> (besucht am 28.07.2025).
- R. Fielding und J. Reschke (Juni 2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. URL: <https://datatracker.ietf.org/doc/html/rfc7231#section-4.2.1> (besucht am 28.07.2025).
- Jakob Jenkov (März 2021). *Multithreading Costs*. URL: <https://jenkov.com/tutorials/java-concurrency/costs.html> (besucht am 28.07.2025).
- kingthorin und zbraiterman (2025). *SQL Injection*. Jahr der Quelle nicht angegeben; Inhalt ist jedoch aktuell. OWASP. URL: https://owasp.org/www-community/attacks/SQL%5C_Injection (besucht am 28.07.2025).
- Chaimaa Lotfi, Swetha Srinivasan, Myriam Ertz und Imen Latrous (Jan. 2021). „Web Scraping Techniques and Applications: A Literature Review“. In: S. 381–394.
- Jeremy Miller (Okt. 2019). *Patterns in Practice: Cohesion And Coupling*. Microsoft. URL: <https://learn.microsoft.com/en-us/archive/msdn-magazine/2008/october/patterns-in-practice-cohesion-and-coupling> (besucht am 28.07.2025).
- MongoDB (2025). *MongoDB Documentation*. Jahr der Quelle nicht angegeben; Inhalt ist jedoch aktuell. MongoDB, Inc. URL: <https://www.mongodb.com/docs/> (besucht am 28.07.2025).
- European Parliament und of the Council (Apr. 2016). *Regulation (EU) 2016/679: Protection of natural persons with regard to the processing of personal data and on the free movement of*

- such data (General Data Protection Regulation)*. Official Jouarnal of the European Union. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (besucht am 28.07.2025).
- Łukasz Ryś (Mai 2024). *Organizing Layers Using Hexagonal Architecture, DDD, and Spring*. Baeldung - Tarnum Java SRL. URL: <https://www.baeldung.com/hexagonal-architecture-ddd-spring> (besucht am 28.07.2025).
- Selenium (2025). *The Selenium Browser Automation Project*. Jahr der Quelle nicht angegeben; Inhalt ist jedoch aktuell. Selenium. URL: <https://www.selenium.dev/documentation/> (besucht am 28.07.2025).
- Jason Struck (Feb. 2025). *Introduction to Web Scraping with R*. University of Wisconsin-Madison, Social Science Computing Cooperative. URL: <https://sscc.wisc.edu/sscc/pubs/web scraping-r/index.html> (besucht am 28.07.2025).
- WHATWG (Juli 2025). 4.9.1 *The table element*. Zuletzt aktualisiert am 28.07.2025. WHATWG. URL: <https://html.spec.whatwg.org/multipage/tables.html%5C#the-table-element> (besucht am 28.07.2025).

Abbildungsverzeichnis

1	Hexagonale Architektur dieser Anwendung	7
---	---	---

Tabellenverzeichnis

1	Attribute der Klasse <code>ScrapedData</code>	8
2	Visualisierung der zugehörigen Tabelle	9
3	Attribute der Klasse <code>AutomatedConfig</code>	9
4	Mögliche Werte der Klasse <code>Timeframe</code>	10
5	Attribute der Klasse <code>SettingsData</code>	11
6	Tabelle mit mehrzeiligem Tabellenkopf	16
7	Übersicht der Aufträge unter <code>/automated</code>	19
8	Übersicht der Aufträge unter <code>/automated/testName1</code>	19
9	Übersicht spezifischer Daten, verändert entnommen aus: https://statisticstimes.com/economy/countries-by-gdp.php	19
10	Beispiele für die automatische Typ-Zuweisung beim Export	20
11	Vertikale Key-Value-Tabelle	23
12	Zugehörige Tabelle im Originalformat	24
13	Tabelle nach erfolgreicher Extraktion durch den Web Scraper	24
14	Zugehörige Visualisierung der Tabelle	25
15	Visualisierung einer solchen Tabelle	26
16	Visualisierung der zugehörigen Daten nach Extraktion	27
17	Passende Aufteilung in Spalten des zugehörigen Codes	28
18	Visualisierung der zugehörigen einfachen Liste	28